

Graduation Project

Deadline: 16th of August, 2025 at 11:59. All team members submit on teams' form.

Graduation Project Title: Smart Environmental Monitoring & Access System

1 About the Project

This project focuses on **real-time monitoring** (*You can choose any idea*) and **access control** using ESP32, sensors, actuators, and cloud services (bonus). It enables data visualization, secure access (bonus), and responsive control using **Supabase (SQL backend)**, **MQTT**, and **web/mobile applications**.

Use Case:

- Monitoring using three sensors.
 - Controls access using **keypad (bonus)** logic.
 - Displays live data on **LCD**.
 - Sends and receives data via **MQTT** to/from a Flutter or Web App.
 - Integrates with **Supabase API**.
 - Supports chatbot querying of system status via **LangChain**. (bonus)
 - Is fully **Dockerized** and **deployed on Azure**.
-

2 Core Components & Requirements (22 Grades)

Component	Description
✓ Actuator	Servo motor controlling a smart lock
✓ 3 Sensors	Any three sensors
✓ Keypad Module (Bonus)	Password entry for smart door access
✓ LCD Display	Shows real-time sensor readings
✓ Maquette	Wooden model showcasing full setup
✓ ESP32 + Sensors Integration	All components connected and tested via Platform IO
✓ MQTT Communication	HiveMQ used for sensor-actuator cloud communication
✓ Supabase via REST API	Replaces Firebase: full read/write of user/sensor info
✓ Web or Flutter App	App includes login, sign-up, dashboard, control, and home screens (≥ 5)

✓ Supabase Auth

Authentication system linked to Supabase auth system

3 Bonus Extensions (Multiple Options)

Bonus Feature

Details

✓ Flask API
(Real-life
Source)

Middleware layer fetching sensor data or
controlling actuators

✓ LangChain
Chatbot Agent

Ask questions like “What’s the gas level now?” –
answers by calling Flask API

✓ Dockerized
System (Bonus)

Web app, API, chatbot, and ESP32 backend run
as separate Docker containers

✓ Azure
Deployment
(Bonus)

Entire system hosted using Azure App Services /
Azure IoT Hub

4 Deliverables

Deliverable	Description
 MP4 Demo (Arabic Voice)	≤6 min video showing all components + narration
 ESP32 C++ Code	Fully commented, functional ESP32 code controlling sensors and MQTT
 Web/Mobile App Code	Web: React or Vanilla JS OR Flutter app with mostly commented code
 Wokwi Circuit	Circuit designed in Wokwi with all connections illustrated
 Presentation (≥15 slides)	Includes overview, tech stack, sensors, logic, architecture, output, and future work
 GitHub Push	Push all code + media to GitHub with README
 Team Worksheet	Describe each member's work and time spent

5 Application Flow

✓ ESP32 Functionality

- Reads sensor data from sensors.
- Displays data on LCD.
- Controls servo motor based on input logic.
- Publishes data to HiveMQ.
- Subscribes to control messages from app.

✓ MQTT Integration

- HiveMQ broker for publish/subscribe:
 - `esp32/sensors/data`
 - `esp32/door/control`
- Node-RED (optional): Can be used to forward to Supabase if needed.

✓ Supabase Integration

- Database tables:
 - `users`, `sensor_readings`, `alerts`, `access_logs`
- REST API used to push sensor values and fetch historical data. (Bonus)
- Supabase Auth for login/signup.

✓ Web/Mobile App Features

- Sign in / Sign up using Supabase auth
- Dashboard to view sensor data in real-time
- Control door remotely (send MQTT)
- Alerts page for gas/fire incidents
- History logs (access/sensor)

✓ Flask API (Bonus)

- Simulates external real-life data (e.g., pull rainfall or pollution index)
- Used by both ESP32 backend and LangChain agent

✓ LangChain Chatbot (Bonus)

- User types: *"What's the temperature now?"*
- LangChain queries the Flask API
- Returns: *"The current temperature is 28.7°C"*

✓ Dockerization (Bonus)

- `esp32-server` (mock MQTT subscriber/publisher)
- `flask-api` (for LangChain + external integration)
- `web-ui` (web app)
- `supabase-postgREST` (API simulator for local use)

✓ Azure Deployment (Bonus)

- Docker containers deployed using Azure App Service or AKS
- Optionally use Azure IoT Hub for full enterprise setup